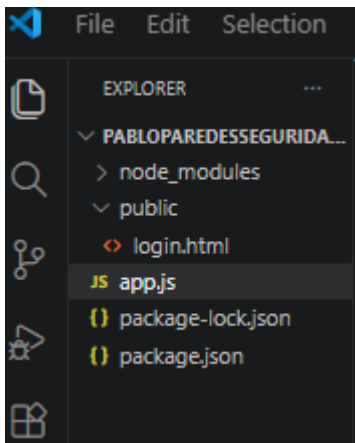


En esta actividad se desarrolló una aplicación web simple utilizando Node.js y Express. Se implementó un sistema de autenticación mediante login, uso de sesiones y cookies, además de una ruta protegida con control de acceso.

TECNOLOGÍAS UTILIZADAS:

- Node.js
- Express.js
- express-session
- Visual Studio Code
- GitHub
- HTML

El proyecto fue organizado de la siguiente manera:



La carpeta public contiene la interfaz del login. El archivo app.js contiene la lógica del servidor Express, autenticación, sesiones y protección de rutas.

Autenticación:

Se implementó un sistema de autenticación simple mediante usuario y contraseña. El sistema valida las credenciales ingresadas antes de permitir el acceso.

Se utilizaron sesiones mediante express-session para mantener el estado del usuario autenticado. Además, se configuraron atributos de seguridad en la cookie:

- HttpOnly
- SameSite

Se creó una ruta protegida llamada /privado, la cual solo puede ser accedida si existe una sesión válida.

HTTPS permite cifrar la comunicación entre cliente y servidor mediante TLS, evitando que terceros puedan interceptar información sensible como credenciales o cookies.

En el proyecto se explicó el uso conceptual de HTTPS y su relación con cookies seguras mediante el atributo Secure.

Inicio de Sesión:

MENU



Iniciar Sesión

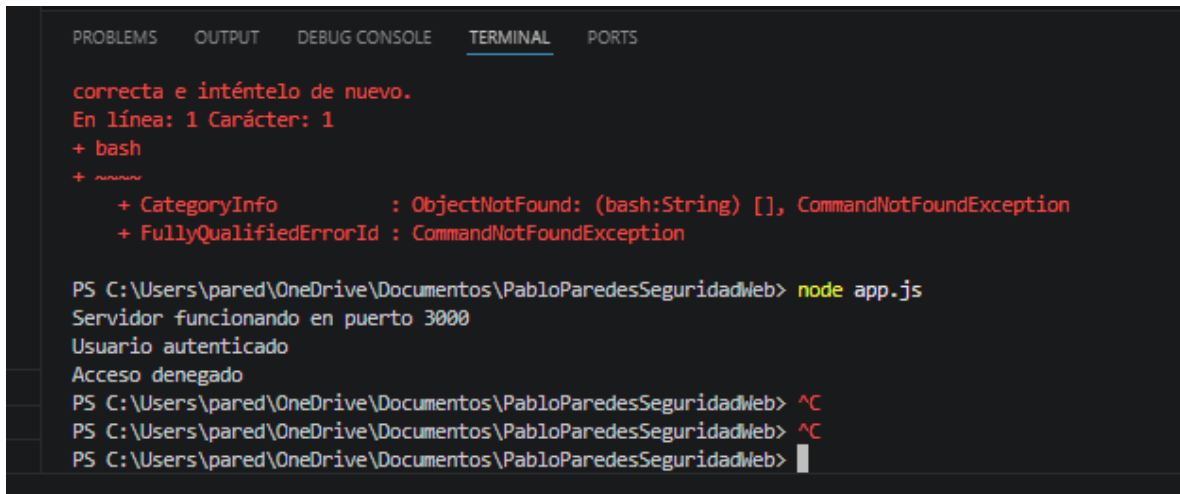
Bienvenido a la zona privada/Login



Credenciales Incorrectas/Fallo



Console



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

correcta e inténtelo de nuevo.
En línea: 1 Carácter: 1
+ bash
+ bash
+ CategoryInfo          : ObjectNotFound: (bash:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\pared\OneDrive\Documentos\PabloParedesSeguridadWeb> node app.js
Servidor funcionando en puerto 3000
Usuario autenticado
Acceso denegado
PS C:\Users\pared\OneDrive\Documentos\PabloParedesSeguridadWeb> ^C
PS C:\Users\pared\OneDrive\Documentos\PabloParedesSeguridadWeb> ^C
PS C:\Users\pared\OneDrive\Documentos\PabloParedesSeguridadWeb> 
```

¿Qué riesgo reduce el uso de HttpOnly y Secure?

HttpOnly evita que scripts JavaScript accedan a las cookies, reduciendo ataques XSS. Secure permite que las cookies solo sean enviadas mediante conexiones HTTPS.

¿Qué diferencia hay entre autenticación y autorización?

La autenticación verifica la identidad del usuario mediante login y contraseña. La autorización controla si el usuario tiene permiso para acceder a ciertos recursos o rutas protegidas.

¿Qué mejorarías para producción?

Si el sistema fuera publicado en producción, se podrían implementar mejoras como:

- Uso obligatorio de HTTPS
- Contraseñas cifradas
- Base de datos real